

Aula 8: Herança

Programação Orientada a Objetos em Java

Nesta aula, exploraremos um dos pilares fundamentais da Orientação a Objetos: a **Herança**. Aprenderemos como classes podem compartilhar comportamentos e atributos, promovendo reutilização de código e organização hierárquica em Java.

POO EM
JAVA

AULA 8



Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

1

Conceito de Herança

Compreender o que é herança e sua importância na programação orientada a objetos.

2

Sintaxe em Java

Aplicar corretamente a palavra-chave `extends` para declarar herança entre classes.

3

Super e Subclasse

Diferenciar superclasse e subclasse, e utilizar a palavra-chave `super` para acessar membros herdados.

4

Polimorfismo

Entender o polimorfismo de inclusão e como ele torna o código mais flexível e extensível.

O Que É Herança?

Herança é o mecanismo da OO que permite que uma classe **herde características** (atributos e métodos) de outra classe já existente. É uma forma poderosa de promover a reutilização de código e organizar sistemas de forma hierárquica.

Relação "É Um"

A subclasse É UM tipo da superclasse. Ex: Cachorro é *um* Animal.

Reutilização

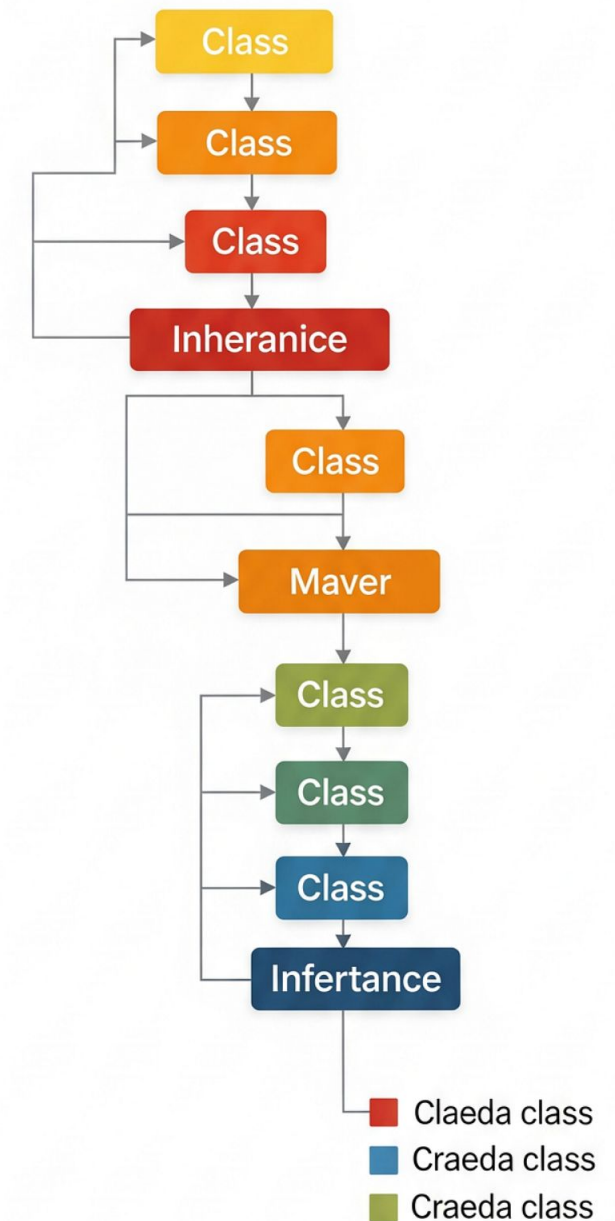
Evita duplicação de código. Atributos e métodos comuns ficam na superclasse.

Extensão

A subclasse pode **adicionar** novos atributos e métodos específicos.

Especialização

A subclasse pode **modificar** comportamentos herdados para sua necessidade.



Terminologia Essencial

Superclasse

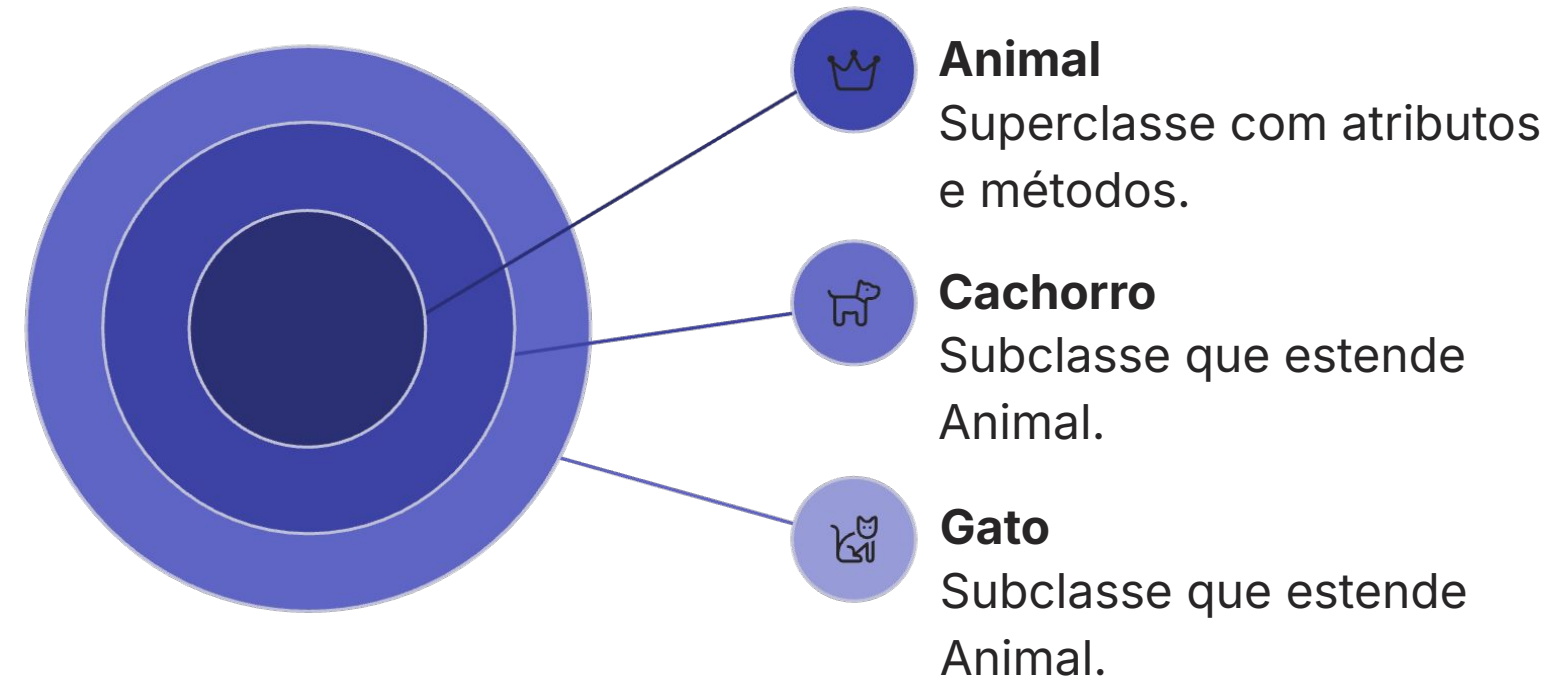
Também chamada de **classe pai, mãe ou base**. É a classe que disponibiliza seus atributos e métodos para ser herdada.

Subclasse

Também chamada de **classe filha ou derivada**. É a classe que herda e pode especializar os membros da superclasse.

Hierarquia de Classes

Estrutura em **árvore** que organiza as classes por nível de generalidade. A raiz em Java é sempre a classe `Object`.



Toda classe em Java herda implicitamente de `Object`, a raiz de toda hierarquia.

Sintaxe de Herança em Java

Em Java, usamos a palavra-chave `extends` para declarar que uma classe herda de outra. A sintaxe é direta e objetiva:

```
// Declaração da superclasse
public class Animal {
    protected String nome;
    protected int idade;

    public void comer() {
        System.out.println(nome + "
está comendo.");
    }
}
```

```
// Declaração da subclasse com extends
public class Cachorro extends Animal {
    private String raca;

    public void buscarBola() {
        System.out.println(nome + "
buscou a bola!");
    }
}
```

▲ Herança Simples

Java permite herdar de **apenas uma** superclasse por vez.

🔒 Construtores

Construtores **não são herdados**, mas podem ser chamados via `super()`.

🚫 Private

Membros `private` não são acessíveis diretamente pela subclasse.

Exemplo Prático: Superclasse Animal

```
// Superclasse
public class Animal {

    // Atributos protected: acessíveis pelas
    subclasses
    protected String nome;
    protected int idade;

    // Construtor
    public Animal(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
}
```

Por que protected?

O modificador `protected` permite que os atributos `nome` e `idade` sejam acessados diretamente pelas subclasses, ao contrário de `private`. Isso é um padrão comum em hierarquias de herança.

- ❑ O método `fazerSom()` é definido na superclasse de forma genérica e será **sobrescrito** em cada subclasse com o som específico do animal.

Exemplo Prático: Superclasse Animal

```
// Métodos gerais
public void comer() {
    System.out.println(nome + " está comendo.");
}

public void dormir() {
    System.out.println(nome + " está dormindo.");
}

// Método que pode ser sobrescrito
public void fazerSom() {
    System.out.println("Som genérico.");
}
}
```

Por que `protected`?

O modificador `protected` permite que os atributos `nome` e `idade` sejam acessados diretamente pelas subclasses, ao contrário de `private`. Isso é um padrão comum em hierarquias de herança.

- ❑ O método `fazerSom()` é definido na superclasse de forma genérica e será **sobrescrito** em cada subclasse com o som específico do animal.

Exemplo Prático: Subclasse Cachorro

```
public class Cachorro extends Animal {  
    // Atributo específico da subclasse  
    private String raca;  
  
    // Construtor chama super()  
    public Cachorro(String nome,  
                     int idade,  
                     String raca) {  
        super(nome, idade); // chama Animal()  
        this.raca = raca;  
    }  
}
```

O que Cachorro herda de Animal?

- **Atributos herdados:** nome e idade (protected)
- **Métodos herdados:** comer() e dormir()
- **Adiciona:** atributo raca e método buscarBola()
- **Sobrescreve:** fazerSom() → "Au au!"

Exemplo Prático: Subclasse Cachorro

```
// Método específico da subclasse
public void buscarBola() {
    System.out.println(nome +
        " buscou a bola!");
}

// Sobrescrita do método fazerSom()
@Override
public void fazerSom() {
    System.out.println("Au au!");
}
}
```

O que Cachorro herda de Animal?

- **Atributos herdados:** nome e idade (protected)
- **Métodos herdados:** comer() e dormir()
- **Adiciona:** atributo raca e método buscarBola()
- **Sobrescreve:** fazerSom() → "Au au!"

A Palavra-Chave `super`

A palavra-chave `super` é usada dentro de uma subclasse para referenciar membros da **superclasse direta**. É essencial para evitar ambiguidades e reaproveitar lógica já implementada.

1

`super.atributo`

Acessa um atributo da superclasse quando há conflito de nomes com a subclasse.

```
super.nome
```

2

`super.metodo()`

Chama um método da superclasse, útil para reaproveitar a implementação original.

```
super.fazerSom();
```

3

`super()`

Chama o construtor da superclasse.

Deve ser a primeira linha do construtor da subclasse.

```
super(nome, idade);
```

📌 **this vs super:** `this` referencia o próprio objeto atual; `super` referencia a superclasse. Ambos não podem ser usados em métodos estáticos.

Chamada de Construtores com `super()`

Construtores **não são herdados**, mas a subclasse *deve* inicializar a parte da superclasse. Isso é feito com `super()`.

```
public class Animal {
    protected String nome;
    protected int idade;

    public Animal(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
        System.out.println("Animal criado: " + nome);
    }
}
```

```
public class Gato extends Animal {
    private String pelagem;
    public Gato(String nome, int idade, String
    pelagem) {
        super(nome, idade); // ← PRIMEIRA INSTRUÇÃO
        OBRIGATÓRIA
        this.pelagem = pelagem;
        System.out.println("Gato criado com pelagem: " +
        pelagem);
    }
}
```

Chamada implícita

Se `super()` não for escrito, Java insere automaticamente a chamada ao construtor **padrão (sem parâmetros)** da superclasse.

Erro comum

Se a superclasse não tem construtor padrão e `super()` não é chamado explicitamente, ocorre **erro de compilação**.

Sobrescrita de Métodos — @Override

Sobrescrita (override) acontece quando a subclasse cria novamente um método que já existe na superclasse, usando **o mesmo nome, os mesmos parâmetros e o mesmo tipo de retorno**. Isso permite que a subclasse tenha um comportamento próprio para esse método.

A anotação **@Override** é muito importante porque ajuda o compilador a verificar se a sobrescrita foi feita corretamente

Regra de visibilidade

O método sobrescrito **não pode ser mais restritivo** que o original. Ex: não pode tornar **public** em **private**.

Assinatura idêntica

Nome, lista de parâmetros e tipo de retorno devem ser **exatamente iguais** aos do método na superclasse.

```
// Na superclasse Animal:  
public void fazerSom() {  
    System.out.println("Som  
genérico de animal.");  
}
```

```
// Na subclasse Cachorro:  
@Override  
public void fazerSom() {  
    System.out.println("Au  
au!");  
}
```

```
// Na subclasse Gato:  
@Override  
public void fazerSom() {  
    System.out.println("Miau!");  
}
```

Polimorfismo de Inclusão

Uma variável do tipo **superclasse** pode referenciar um objeto de qualquer subclasse. Isso é o **polimorfismo de inclusão** — o método executado é sempre o da classe real do objeto (ligação dinâmica).

```
// Variável do tipo Animal referenciando objetos  
de subclasses
```

```
Animal a1 = new Cachorro("Rex", 3, "Labrador");  
Animal a2 = new Gato("Mimi", 2, "Rajada");
```

```
// O método chamado é o da subclasse real  
(ligação dinâmica)
```

```
a1.fazerSom(); // Imprime: "Au au!"  
a2.fazerSom(); // Imprime: "Miau!"
```

```
// Array polimórfico
```

```
Animal[] animais = { new Cachorro("Rex", 3,  
"Labrador"), new Gato("Mimi", 2, "Rajada") };
```

```
for (Animal a : animais) {  
    a.fazerSom(); // Cada animal faz seu som  
    específico  
}
```

Código Flexível

Novos tipos de Animal podem ser adicionados sem alterar o código existente.

Ligação Dinâmica

Java decide qual método executar em **tempo de execução**, não em compilação.

Base para Padrões

Fundação de padrões de projeto como **Strategy** e **Template Method**.

Tipo Estático vs. Tipo Dinâmico

```
// Tipo estático: Animal (declarado)
// Tipo dinâmico: Cachorro (real)
Animal a = new Cachorro("Rex", 3, "Labrador");

// Compilador vê: Animal
// JVM executa: Cachorro.fazerSom()
a.fazerSom(); // "Au au!"

// ATENÇÃO: método buscarBola() NÃO
// está disponível via tipo estático Animal
// a.buscarBola(); // ERRO de compilação!

// Para acessar, é necessário cast:
Cachorro c = (Cachorro) a;
c.buscarBola(); // OK!
```

Entendendo a Diferença

Tipo Estático

Declarado na variável. Verificado em **tempo de compilação**. Define quais métodos podem ser chamados.

Tipo Dinâmico

Tipo real do objeto na memória. Determinado em **tempo de execução**. Define qual implementação do método é executada.

- ❏ Este conceito é fundamental para entender como o polimorfismo funciona "por baixo dos panos" na JVM.

Operador instanceof

O operador `instanceof` verifica, em **tempo de execução**, se um objeto é instância de uma determinada classe ou suas subclasses. É essencial para realizar **casts seguros**.

```
Animal animal = new Cachorro("Rex", 3, "Labrador");

// Verificação de tipo em tempo de execução
if (animal instanceof Cachorro) {
    // Cast seguro após verificação
    Cachorro c = (Cachorro) animal;
    c.buscarBola(); // Agora é seguro chamar!
    System.out.println("É um cachorro!");
}
```

```
if (animal instanceof Gato) {
    System.out.println("É um gato!"); // Não será
    impresso
}

// Sempre verdadeiro: todo Cachorro é um Animal
System.out.println(animal instanceof Animal); //
true
```

Retorno booleano

Retorna `true` se o objeto é instância da classe verificada (ou subclasse dela).

Evita ClassCastException

Sem verificar com `instanceof`, um cast inválido lança `ClassCastException` em tempo de execução.

Hierarquia respeitada

Um `Cachorro` é `instanceof Animal` e também `instanceof Object`.

Boas Práticas e Erros Comuns

✓ Use herança para "é um"

Herança é adequada quando a subclasse realmente **é um tipo** da superclasse. Para relação "tem um" (composição), prefira atributos.

⚠ Prefira composição

Não abuse da herança. **Composição** é muitas vezes mais flexível e menos acoplada do que hierarquias profundas de herança.

⊘ Hierarquias rasas

Evite hierarquias muito profundas (mais de 3-4 níveis). Elas tornam o código difícil de entender e manter.

📐 Princípio de Liskov

Uma subclasse deve poder **substituir** a superclasse sem quebrar o comportamento esperado do programa (LSP).

- ❏ **Erro clássico:** usar herança apenas para reaproveitar código sem que exista uma real relação "é um". Ex: herdar de `ArrayList` só para ter seus métodos — use composição!

Exercícios de Fixação

Coloque em prática o que aprendemos nesta aula! Implemente as hierarquias abaixo em Java e experimente os conceitos de herança, sobrescrita e polimorfismo:

01

Hierarquia Veículo

Crie a superclasse `Veiculo` com atributos `marca` e `velocidade`, e as subclasses `Carro` e `Moto` com atributos e métodos específicos.

02

Sobrescrita de Método

Implemente o método `acelerar()` na superclasse e sobrescreva-o em `Carro` e `Moto` com comportamentos diferentes. Use `@Override`.

03

Uso de `super()`

Use `super(marca, velocidade)` nos construtores de `Carro` e `Moto` para inicializar os atributos herdados de `Veiculo`.

04

Array Polimórfico

Crie um array do tipo `Veiculo[]`, popule com instâncias de `Carro` e `Moto`, e chame `acelerar()` em loop para ver o polimorfismo em ação.

05

Próxima Aula

Revise os conceitos de herança e prepare-se para a **Aula 9: Polimorfismo** — aprofundaremos ligação dinâmica, classes abstratas e interfaces.